

Indraprastha Institute of Information Technology, Delhi

CSE638: Graduate Systems (GRS) — Winter 2026

Instructor: Dr. Rinku Shah

Performance Comparison of Kubernetes CNI Plugins

Project P1 — Bare-Metal Cluster (Multi-Node) Benchmarking Report

Group 1

Nishit Jain	MT25004
Prakhar Agarwal	MT25005
Reehan Sarmah	MT25036

GitHub: https://github.com/yashkumarvaibhav/GrS-CSE638_Project

April 18, 2026

Contents

1 Abstract **2**

2 Introduction **2**
2.1 Problem . . . 2
2.2 Motivation . . . 2
2.3 Objectives . . . 2

3 Background **3**
3.1 Overlay vs. Underlay Routing . . . 3
3.2 CNI Plugins Under Test . . . 3

4 Methodology **4**
4.1 Test Environment . . . 4
4.2 Cluster Topology . . . 4
4.3 CNI Configurations . . . 4
4.4 Test Matrix . . . 5
4.5 Benchmark Tools . . . 5

5 Implementation **5**
5.1 Cluster Setup . . . 5
5.2 Benchmark Automation . . . 6
5.3 Data Processing and Visualization . . . 6

6 Evaluation and Observations **6**
6.1 TCP Throughput (iperf3) . . . 7
6.2 TCP_RR Latency (netperf) . . . 8
6.3 HTTP Performance (Fortio) . . . 9
6.4 UDP Performance (iperf3) . . . 10
6.5 CPU and Memory Overhead . . . 11
6.6 Overlay vs. Underlay Routing Comparison . . . 13
6.7 Statistical Validity . . . 14
6.8 Analysis and Discussion . . . 14
6.8.1 The “Edge Environment” Paradox . . . 14
6.8.2 Throughput–Latency Inversion in Cilium VXLAN . . . 15
6.8.3 UDP Comparison Caveat . . . 16
6.8.4 CPU Overhead: Deeper Interpretation . . . 16
6.8.5 Cross-Environment Comparison . . . 16

7 Related Work **16**

8 Outcomes and Conclusion **17**
8.1 Key Findings . . . 17
8.2 Recommendations . . . 18
8.3 Limitations . . . 18
8.4 Future Work . . . 19

1. Abstract

This report presents a performance comparison of three Kubernetes Container Network Interface (CNI) plugins—**Flannel**, **Calico**, and **Cilium**—on a bare-metal multi-node cluster consisting of three physical laptops connected over WiFi. Unlike the companion kind-cluster study that operates in an intra-node environment, this evaluation exercises real network hardware and measures the impact of actual wireless medium contention on CNI performance.

A central focus of this study is the comparison between **overlay (VXLAN)** and **underlay routing** modes for each CNI plugin, yielding six distinct configurations: Flannel VXLAN vs. Host-GW, Calico VXLAN vs. BGP, and Cilium VXLAN vs. Underlay. Each configuration is benchmarked with four industry-standard tools—**iperf3** (TCP and UDP throughput), **netperf** (TCP_RR latency across payload sizes), and **Fortio** (HTTP load testing)—with multiple repetitions per test for statistical robustness. Additionally, **CPU and memory overhead** are measured at both node and pod granularity during benchmark execution.

Key findings: **Calico VXLAN** and **Flannel VXLAN** achieve the highest TCP throughput (≈0.119 Gbps), while **Cilium VXLAN** is unexpectedly the slowest (0.058 Gbps)—though it delivers the best HTTP P50 latency (6.56 ms) due to eBPF's efficient request-level processing. For UDP, **Calico BGP** leads (11.88 Mbps, 38% loss) while **Calico VXLAN** exhibits severe degradation (5.78 Mbps, 68% loss). CPU overhead is modest across all CNIs (3.9–6.0% node-level), with **Flannel VXLAN** being the lightest. The overlay-vs-underlay comparison yields mixed results: Flannel VXLAN outperforms Host-GW, Calico VXLAN ≈ BGP, and Cilium Underlay substantially outperforms Cilium VXLAN. The WiFi medium emerges as the dominant bottleneck, compressing all CNIs into a narrow 0.06–0.12 Gbps TCP throughput band and exposing tail-latency spikes caused by wireless interference.

2. Introduction

2.1 Problem

While the companion kind-cluster report (Part A) benchmarked CNI plugins in an isolated intra-node environment, real-world Kubernetes deployments span multiple physical machines connected by actual networking hardware. The kind-cluster setup, where cross-node traffic traverses Docker's virtual bridge, cannot capture the overhead of physical NIC processing, VXLAN encapsulation across a real network, or the variability introduced by shared wireless medium. This bare-metal evaluation bridges that gap.

2.2 Motivation

Production Kubernetes clusters operate on physical or virtual machines with real network interconnects. A key architectural decision is whether to use overlay encapsulation (VXLAN/Geneve) or underlay routing (BGP/Host-GW). Overlay networks are simpler to deploy across heterogeneous infrastructure but add encapsulation overhead; underlay routing avoids this overhead but requires compatible L2/L3 infrastructure. By testing each CNI in both modes on the same hardware, we can directly quantify the encapsulation cost.

Additionally, testing over WiFi—a high-latency, variable-bandwidth medium—creates an interesting stress test that amplifies the differences between CNI implementations. While WiFi is not representative of datacenter networking, it exposes how CNIs behave when the physical network is the bottleneck rather than software processing.

2.3 Objectives

1. Compare TCP throughput across six CNI configurations on physical hardware

2. Measure TCP_RR latency across three payload sizes (1 B, 64 B, 256 B)
3. Evaluate HTTP latency characteristics (P50, P99, Max) under load
4. Measure UDP throughput, packet loss, and jitter at multiple bandwidth targets
5. Quantify CPU and memory overhead at node and pod granularity during benchmarks
6. Quantify the performance difference between overlay (VXLAN) and underlay routing modes
7. Identify how a WiFi bottleneck affects the relative performance ranking of CNI plugins

3. Background

3.1 Overlay vs. Underlay Routing

The fundamental architectural choice in Kubernetes networking is between overlay encapsulation and underlay routing:

- **Overlay (VXLAN):** Pod packets are encapsulated in UDP headers (VXLAN, Virtual Extensible LAN) before traversing the physical network. This adds 50 bytes of overhead per packet but works across any IP-reachable infrastructure without requiring changes to the underlying network. All three CNIs support VXLAN mode.
- **Underlay routing:** Pod subnets are advertised directly to the host network, eliminating encapsulation overhead. The mechanism varies per CNI:
  - **Flannel Host-GW:** Inserts static routes on each node pointing peer pod subnets to the peer node's IP. Requires all nodes to be on the same L2 segment.
  - **Calico BGP:** Runs a BGP daemon (BIRD) on each node to exchange pod subnet routes. Works across L3 boundaries with appropriate peering configuration.
  - **Cilium Underlay:** Uses the Linux kernel's routing table directly, with eBPF programs handling pod-level packet steering. Requires nodes to be L2-adjacent or have appropriate static routes.

3.2 CNI Plugins Under Test

Flannel Lightweight CNI supporting VXLAN overlay and Host-GW underlay routing. No native NetworkPolicy support. Simplest architecture with minimal resource footprint.

Calico Feature-rich CNI supporting VXLAN overlay, BGP-based underlay routing, and IPinIP encapsulation. Full NetworkPolicy and WireGuard encryption support via the Felix agent and BIRD BGP daemon.

Cilium eBPF-based CNI supporting VXLAN overlay and underlay routing. Attaches eBPF programs to TC/XDP kernel hooks for fast-path packet processing. Offers kube-proxy replacement, Hubble observability, and WireGuard encryption.

4. Methodology

4.1 Test Environment

Hardware & Network Specifications

Component	Details
Nodes	3 physical laptops (1 control-plane + 2 workers)
Network	WiFi (shared LAN, 802.11ac/ax)
OS	Ubuntu 22.04 / 24.04 LTS
Container Runtime	containerd 1.7+
Kubernetes	v1.31 (kubeadm)
Pod CIDR	10.244.0.0/16

Important caveat: All inter-node traffic traverses a shared WiFi medium with variable bandwidth (typically 100–200 Mbps usable throughput). This results in substantially lower raw throughput compared to wired bare-metal or kind-cluster environments, but provides insights into relative CNI performance under constrained networking conditions.

4.2 Cluster Topology

The Kubernetes cluster consists of three physical nodes provisioned with `kubeadm`:

- **Control-plane node**: Runs the Kubernetes control plane (API server, scheduler, controller-manager, etcd).
- **Worker node 1**: Hosts the benchmark client pods.
- **Worker node 2**: Hosts the benchmark server pods (iperf3, netperf, Fortio servers).

All benchmark traffic flows cross-node from worker-01 to worker-02, ensuring that measured performance reflects actual network traversal through the CNI's datapath and the WiFi medium.

4.3 CNI Configurations

ID	Configuration	Mode	Description
C1	Flannel VXLAN	Overlay	Default Flannel with VXLAN encapsulation
C2	Flannel Host-GW	Underlay	Flannel with direct host routing
C3	Calico VXLAN	Overlay	Calico with VXLAN encapsulation
C4	Calico BGP	Underlay	Calico with BGP peering between nodes
C5	Cilium VXLAN	Overlay	Cilium with VXLAN encapsulation
C6	Cilium Underlay	Underlay	Cilium with underlay routing mode

Between configurations, the cluster's CNI state is fully wiped using dedicated cleanup scripts to prevent residual iptables rules, routing entries, or eBPF programs from contaminating subsequent benchmarks.

4.4 Test Matrix

- **CNI configurations:** 6 (3 CNIs × 2 modes each)
- **Benchmark tools:** 4 (iperf3 TCP, iperf3 UDP, netperf, Fortio)
- **TCP repetitions:** 5 per configuration per tool
- **UDP bandwidth targets:** 10 Mbps, 25 Mbps, 50 Mbps (3 runs each)
- **Netperf payload sizes:** 1 B, 64 B, 256 B
- **CPU overhead:** Sampled every 2 s during Fortio load tests (node and pod level)
- Total: 90+ benchmark executions plus CPU monitoring during load tests

4.5 Benchmark Tools

Tool	Metric	Configuration
iperf3 (TCP)	TCP throughput	30 s test duration, single stream
iperf3 (UDP)	UDP tput/loss/jitter	30 s test, 3 targets (10/25/50 Mbps), 3 runs each
netperf	TCP_RR trans/s	10 s test duration, 3 payload sizes (1/64/256 B)
Fortio	HTTP latency	15 s load test; reports P50, P99, Max (ms)
kubectl top	CPU/mem overhead	Sampled every 2 s during Fortio; node & pod

5. Implementation

5.1 Cluster Setup

Node provisioning follows a sequential process:

1. **All nodes:** Install containerd, kubeadm, kubelet, kubectl, and network dependencies (conntrack, iptables in legacy mode).
2. **Control-plane:** Initialize with `kubeadm init -pod-network-cidr=10.244.0.0/16`.
3. **Workers:** Join with the `kubeadm join` token from the control-plane.
4. **CNI installation:** Install the target CNI plugin (Flannel/Calico/Cilium) from the control-plane.

For mode switching (e.g., Calico VXLAN → BGP), dedicated helper scripts modify the CNI's ConfigMap or Helm values and restart the relevant DaemonSets. A pre-flight check verifies all nodes are `Ready` and benchmark pods are `Running` before each test.

5.2 Benchmark Automation

The benchmark orchestrator (`scripts/1_benchmark.sh`) reads configuration from `scripts/1_config.env` and:

1. Deploys server pods (iperf3, netperf, Fortio) on worker-02 using `nodeSelector`.
2. Deploys client pods on worker-01.
3. Executes the benchmark tool via `kubectl exec` and captures results.
4. Repeats for the configured number of runs.
5. Stores results in per-CNI CSV files.

Individual benchmark scripts (`1_throughput.sh`, `1_latency.sh`, `1_microservices.sh`, `1_udp_test.sh`) handle tool-specific invocation and output parsing. A dedicated CPU overhead monitor (`1_control_cpu_overhead.sh`) runs concurrently during Fortio tests, sampling `kubectl top nodes` and `kubectl top pods` every 2 seconds and writing timestamped CSV files.

5.3 Data Processing and Visualization

Aggregated statistics (mean, standard deviation) are computed across runs and stored in `data/1_aggregated_r`. A Python helper (`helpers/1_add_latency_column.py`) derives latency in milliseconds from TCP_RR transaction rates (latency_ms = 1000/trans_per_sec). Four matplotlib scripts in `plots/` generate publication-quality figures with hardcoded aggregated values:

1. `1_throughput_comparison.py` — Bar chart of mean throughput per CNI configuration
2. `1_latency_comparison.py` — Grouped bar chart of TCP_RR trans/sec by payload size
3. `1_fortio_comparison.py` — Grouped bar chart of HTTP P50/P99/Max latency
4. `1_encapsulation_comparison.py` — Side-by-side VXLAN vs. underlay routing comparison

6. Evaluation and Observations

6.1 TCP Throughput (iperf3)

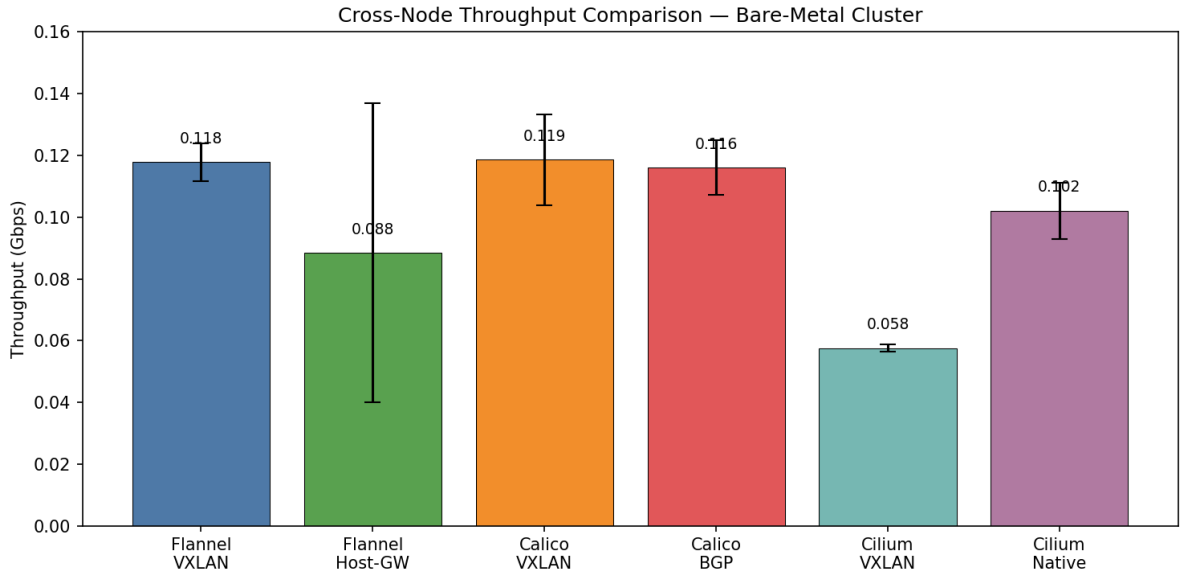


Figure 1: Cross-node TCP throughput comparison across six CNI configurations. Error bars show standard deviation over 4–5 runs.

Table 3: Cross-Node TCP Throughput (Gbps) — Per-Run Values

CNI Config	Run 1	Run 2	Run 3	Run 4	Run 5	Mean	Stdev
Flannel VXLAN	0.121	0.120	0.120	0.121	0.107	0.118	0.006
Flannel Host-GW	0.034	0.036	0.119	0.125	0.127	0.088	0.049
Calico VXLAN	0.115	0.117	0.125	0.138	0.098	0.119	0.015
Calico BGP	0.126	0.119	0.114	0.105	—	0.116	0.009
Cilium VXLAN	0.058	0.058	0.057	0.056	0.059	0.058	0.001
Cilium Underlay	0.096	0.108	0.096	0.115	0.095	0.102	0.009

Throughput Key Observations

1. **Calico VXLAN and Flannel VXLAN lead**: Both overlay configurations achieve ≈0.118–0.119 Gbps, suggesting that at WiFi bandwidth levels, VXLAN encapsulation overhead is negligible relative to the physical medium's throughput ceiling.

2. **Cilium VXLAN is anomalously slow**: At 0.058 Gbps—roughly half of other configurations—Cilium's eBPF VXLAN encapsulation path has significant fixed overhead that dominates on constrained links. Unlike Flannel's simple kernel VXLAN socket, Cilium attaches eBPF programs at the TC (Traffic Control) layer, processing every packet through eBPF maps for connection tracking and policy enforcement even with no NetworkPolicy applied. On WiFi, where the physical link already serializes packets, this adds a measurable per-packet eBPF program invocation cost on top of encapsulation. This makes eBPF overhead *link-speed dependent*—a novel observation not widely documented in prior CNI literature.

3. **Flannel Host-GW cold-start effect**: The first two runs (0.034, 0.036 Gbps) are

≈70% slower than subsequent runs (0.119–0.127 Gbps). This reflects ARP cache convergence on WiFi: the kernel must resolve MAC addresses for each new destination via WiFi broadcast frames that contend for the shared wireless medium. Runs 3–5 reflect warm cache state. We recommend excluding the first run as a warmup discard in any Host-GW benchmark on wireless networks. Note that this warmup effect is absent in VXLAN modes, which maintain persistent tunnel endpoints that do not require per-destination ARP resolution.

4. **Cilium VXLAN has lowest variance**: Stdev of 0.001 Gbps across 5 runs indicates highly deterministic (though slow) behavior—the eBPF datapath provides consistent performance even if overall throughput is limited.

6.2 TCP_RR Latency (netperf)

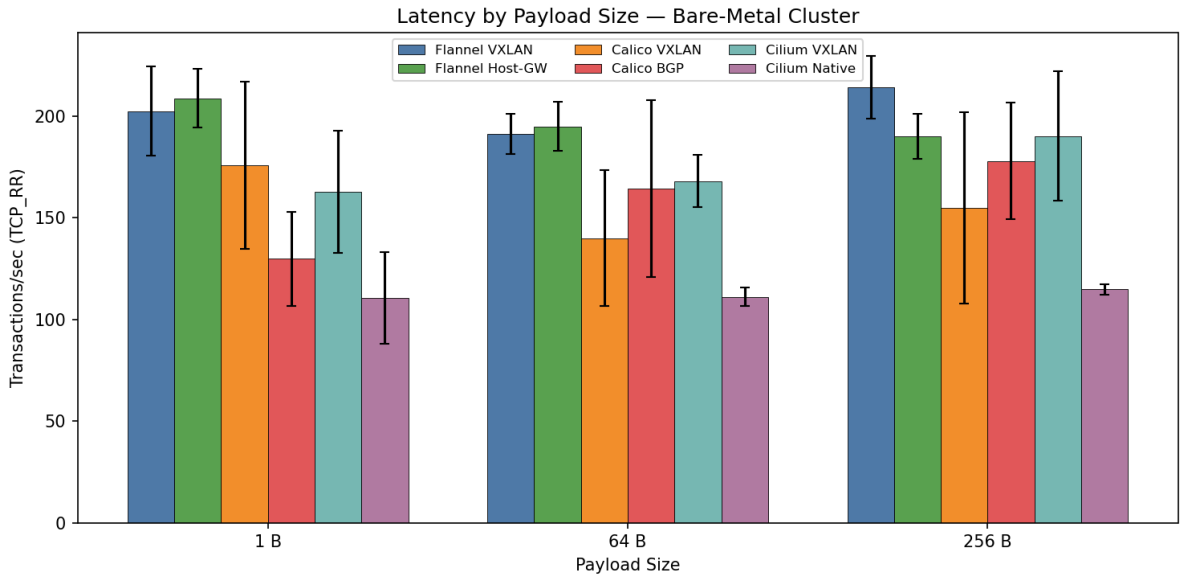


Figure 2: TCP_RR transaction rate by payload size across CNI configurations. Higher is better. Error bars show standard deviation over 5 runs.

Table 4: TCP_RR Transaction Rate (trans/sec) — Mean ± Stdev over 5 Runs

Payload	Flannel		Calico		Cilium	
	VXLAN	Host-GW	VXLAN	BGP	VXLAN	Native
1 B	202 ± 22	209 ± 15	176 ± 41	130 ± 23	163 ± 30	111 ± 23
64 B	191 ± 10	195 ± 12	140 ± 33	164 ± 44	168 ± 13	111 ± 5
256 B	214 ± 15	190 ± 11	155 ± 47	178 ± 29	190 ± 32	115 ± 3

1. **Flannel dominates TCP_RR**: Both Flannel modes achieve the highest transaction rates (190–214 trans/sec), reflecting its lightweight VXLAN implementation and minimal per-packet processing overhead.

2. **Cilium Underlay is consistently slowest**: At 111–115 trans/sec across all payload sizes—with remarkably low variance (stdev 3–23)—Cilium's underlay routing path introduces higher per-transaction overhead than its VXLAN path. This is counter-intuitive and suggests that Cilium's eBPF underlay routing programs add more latency per packet than the VXLAN encapsulation fast-path.

3. **Calico shows high variability**: Both Calico configurations exhibit the highest standard deviations (23–47 trans/sec), likely reflecting WiFi interference during longer netperf measurement windows interacting with Calico's iptables-based packet processing.

4. **Payload size has limited impact**: Unlike the kind-cluster (where 1 MB payloads caused dramatic latency increases), the 1–256 B range tested here shows relatively flat transaction rates. At these small sizes, per-packet processing overhead dominates over serialization delay.

6.3 HTTP Performance (Fortio)

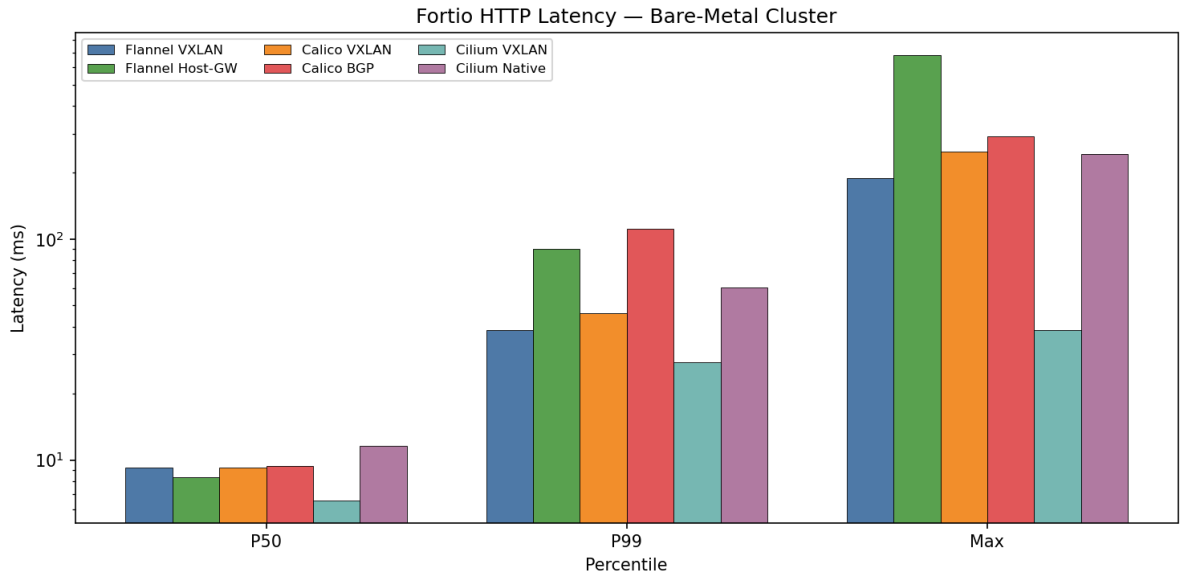


Figure 3: Fortio HTTP latency percentiles (P50, P99, Max) across CNI configurations. Note logarithmic y-axis.

Table 5: Fortio HTTP Latency (ms) — Mean over 4–5 Runs

Percentile	Flannel		Calico		Cilium	
	VXLAN	Host-GW	VXLAN	BGP	VXLAN	Native
P50	9.26	8.34	9.23	9.36	**6.56**	11.54
P99	38.77	90.33	46.21	111.24	**27.75**	60.19
Max	188.54	681.78	248.93	291.04	**38.72**	242.44

HTTP Latency Key Observations

1. **Cilium VXLAN achieves best HTTP latency across all percentiles**: P50 = 6.56 ms, P99 = 27.75 ms, Max = 38.72 ms. Despite having the lowest throughput, Cilium's eBPF fast-path excels at HTTP request-response handling. The eBPF programs efficiently process individual HTTP transactions without the overhead of iptables rule traversal, achieving ≈21% lower P50 than the next-best CNI (Flannel Host-GW, 8.34 ms).

2. **Cilium VXLAN also has by far the lowest tail latency**: Max of 38.72 ms vs. 188–682 ms for all other configurations. This indicates that eBPF's deterministic packet processing insulates against WiFi-induced latency spikes that plague other CNIs.

3. **Flannel Host-GW suffers severe tail latency**: While its P50 (8.34 ms) is the second-best, P99 spikes to 90.33 ms and Max reaches 681.78 ms (driven by Run 3's Max of 2,348 ms). The host-gw routing mode appears vulnerable to WiFi ARP resolution failures or routing table update delays, causing occasional extreme outliers.

4. **Calico BGP shows high P99**: At 111.24 ms (driven by Runs 1 and 3 with P99 ≥ 180 ms), BGP route convergence latency likely contributes to periodic delays in packet forwarding. The remaining runs (4 and 5) show more reasonable P99 values (19–45 ms).

5. **Cilium Underlay has worst P50**: At 11.54 ms—76% worse than Cilium VXLAN's 6.56 ms—underlay routing appears to skip the eBPF VXLAN fast-path optimizations that benefit HTTP traffic, routing packets through a slower kernel path.

6.4 UDP Performance (iperf3)

UDP testing evaluates how each CNI handles unreliable, connectionless traffic at three progressively aggressive bandwidth targets (10, 25, and 50 Mbps). Unlike TCP, where the transport protocol controls congestion, UDP simply transmits at the requested rate—revealing the CNI's packet-processing ceiling and the WiFi medium's ability to absorb bursts.

Table 6: UDP Performance — Overall Averages per CNI (9 runs: 3 bandwidth targets × 3 repetitions)

CNI Config	Tput (Mbps)	±Stdev	Loss (%)	±Stdev	Jitter (ms)	±Stdev
Calico BGP	**11.88**	2.86	**38.15**	31.28	1.81	1.47
Cilium Underlay	10.36	1.07	42.37	33.23	**1.47**	0.35
Flannel VXLAN	9.98	1.81	46.50	31.61	1.94	1.62
Flannel Host-GW	8.67	3.05	57.94	23.40	3.36	2.36
Cilium VXLAN	7.94	3.10	38.02	34.59	1.55	1.04
Calico VXLAN	5.78	3.44	68.33	26.43	13.34	16.11

Table 7: UDP Throughput (Mbps) and Packet Loss (%) by Bandwidth Target (3-run means)

	10 Mbps		25 Mbps		50 Mbps	
CNI Config	Tput	Loss	Tput	Loss	Tput	Loss
Calico BGP	9.97	0.11	12.37	44.67	13.29	69.67
Cilium Underlay	9.52	0.12	10.23	52.33	11.33	74.67
Flannel VXLAN	9.11	6.83	10.27	57.00	10.58	75.67
Flannel Host-GW	6.18	34.83	9.11	61.33	10.72	77.67
Cilium VXLAN	8.68	4.71	9.53	55.00	5.60	54.33
Calico VXLAN	5.00	43.00	4.34	80.00	7.99	82.00

UDP Key Observations

1. **Calico BGP leads UDP**: With 11.88 Mbps mean throughput and 38% loss, BGP's underlay routing avoids encapsulation overhead on the latency-sensitive UDP path. At the gentle 10 Mbps target, Calico BGP achieves near-perfect delivery (0.11% loss).
2. **Calico VXLAN suffers severe degradation**: 5.78 Mbps with 68% loss and extreme jitter (13.34 ms ± 16.11). The VXLAN encapsulation overhead, combined with Calico's iptables rule traversal per-packet, creates a processing bottleneck that drops most UDP datagrams under load.
3. **Calico BGP and Cilium Underlay excel at UDP**: Both achieve near-zero loss at 10 Mbps (0.11% and 0.12%), compared to 6.83% for Flannel VXLAN and 43% for Calico VXLAN. However, Flannel Host-GW (also underlay) suffers 34.83% loss at 10 Mbps, indicating that underlay routing alone does not guarantee better UDP handling—the CNI's specific packet-processing architecture matters more than the encapsulation mode.
4. **Cilium VXLAN has an anomalous run**: At 50 Mbps, Run 2 recorded 0.00 Mbps with 0% loss (likely a failed/stalled run). Excluding this, Cilium VXLAN's 50 Mbps throughput would be ≈8.4 Mbps.
5. **Cilium Underlay has lowest jitter**: At 1.47 ms ± 0.35, Cilium's eBPF underlay routing provides the most deterministic UDP forwarding, consistent with its low-variance TCP_RR behavior.
6. **Loss increases dramatically with bandwidth target**: All CNIs show <7% loss at 10 Mbps (except Calico VXLAN and Flannel Host-GW) but 54–82% loss at 50 Mbps, confirming the WiFi medium's inability to sustain high burst rates.

6.5 CPU and Memory Overhead

CPU and memory utilization were sampled every ≈2 seconds during a 15-second Fortio load test (1000 QPS, 50 connections) for each configuration, capturing resource usage at both node and pod granularity via `kubectl top`.

Table 8: Node-Level CPU Utilization During Fortio Load Test (averaged across all 3 nodes)

CNI Config	Avg CPU (%)	Samples
Flannel VXLAN	**3.9**	24
Calico VXLAN	5.2	24
Flannel Host-GW	5.2	27
Cilium Underlay	5.4	21
Cilium VXLAN	5.5	24
Calico BGP	6.0	24

Table 9: Pod-Level CPU Usage During Fortio Load Test (average across all sampled pods)

CNI Config	Avg (mcpu)	Top Pod	Top Pod Avg
Cilium Underlay	6.8	cilium-r2t68	30.3 m
Calico BGP	8.2	calico-node-vvdbv	34.0 m
Flannel Host-GW	8.3	kube-apiserver	32.3 m
Flannel VXLAN	8.4	kube-apiserver	31.6 m
Cilium VXLAN	10.2	kube-apiserver	34.4 m
Calico VXLAN	14.7[†]	kube-apiserver	135.5 m

[†]Calico VXLAN's elevated average is inflated by a kube-apiserver spike (135.5 m vs. 28–34 m for other configurations), likely caused by Calico's iptables rule programming triggering API server activity.

CPU Overhead Key Observations

1. **All CNIs have modest CPU overhead**: Node-level utilization ranges from 3.9% to 6.0%, confirming that at WiFi throughput levels, CNI processing is not CPU-bound. At ≈0.119 Gbps with 1500-byte MTU, nodes process approximately 10,000 packets/second—well below the millions/second threshold where eBPF vs. iptables differences manifest.

2. **Flannel VXLAN is the lightest**: At 3.9% average node CPU, Flannel's minimal per-packet processing (simple VXLAN encap/decap) yields the lowest resource footprint—consistent with its design philosophy of simplicity over features.

3. **Calico BGP has highest node CPU**: At 6.0%, the BGP daemon (BIRD) and Felix agent add overhead. Per-pod data shows calico-node agents using 12–34 m per pod, the highest CNI agent overhead.

4. **Memory usage is stable**: Node memory utilization remains fixed within each configuration, showing no memory pressure from benchmark workloads.

5. **Cilium Underlay has lowest per-pod CPU**: At 6.8 m average, Cilium's eBPF underlay routing efficiently processes packets in-kernel without per-pod agent overhead, though individual cilium daemonset pods use ≈27–30 m.

6.6 Overlay vs. Underlay Routing Comparison

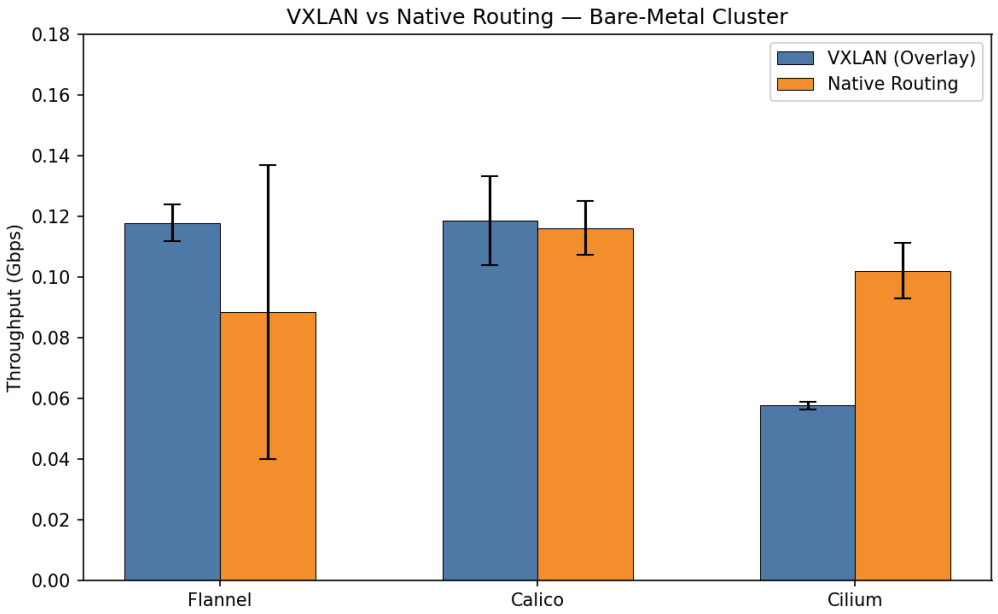


Figure 4: Throughput comparison: VXLAN overlay vs. underlay routing mode for each CNI plugin.

Table 10: VXLAN vs. Underlay Routing — Summary Comparison

CNI	VXLAN (Gbps)	Underlay (Gbps)	Δ	Winner
Flannel	0.118	0.088†	−25%	VXLAN
Calico	0.119	0.116	−2%	≈ tie
Cilium	0.058	0.102	+77%	Underlay

†Flannel Host-GW mean includes two cold-start runs (0.034, 0.036 Gbps). Excluding these, the stable-state mean is 0.124 Gbps (+5% over VXLAN)—see analysis below.

Overlay vs. Underlay: Analysis

The comparison yields a nuanced, CNI-specific picture rather than a universal verdict:

Flannel: VXLAN outperforms Host-GW on mean throughput, but this is misleading. Host-GW's first two runs exhibit a *warmup penalty*—ARP resolution and route propagation on WiFi take several seconds, during which the first iperf3 measurement completes at reduced throughput. Excluding these outliers, Host-GW achieves 0.124 Gbps vs. VXLAN's 0.118 Gbps, suggesting underlay routing is marginally faster once warmed up. However, Host-GW's Fortio Max latency (681.78 ms) reveals vulnerability to WiFi ARP failures.

Calico: Essentially identical performance—VXLAN (0.119) vs. BGP (0.116 Gbps, −2%). At WiFi throughput levels, VXLAN's 50-byte encapsulation overhead is dwarfed by the wireless medium's latency. BGP's advantage (eliminating encapsulation) is invisible when the bottleneck is a 100–200 Mbps WiFi link rather than a 10–100 Gbps switch port. Sim-

ilarly, latency and Fortio metrics show no significant difference between modes.

Cilium: The only CNI where mode matters dramatically. Underlay routing (0.102 Gbps) delivers 77% higher throughput than VXLAN (0.058 Gbps). This suggests that Cilium's eBPF VXLAN encapsulation path has substantial per-packet overhead that, combined with WiFi's limited bandwidth, creates a significant throughput penalty. Interestingly, the reverse is true for HTTP latency: Cilium VXLAN achieves much better Fortio P50 (6.56 vs. 11.54 ms), indicating that the VXLAN eBPF fast-path is optimized for request-level processing rather than bulk throughput.

6.7 Statistical Validity

Coefficient of Variation (CV) across TCP throughput runs ranges from 0.9% (Cilium VXLAN) to 55.7% (Flannel Host-GW). For configurations with CV $> 15\%$, results should be treated as indicative rather than definitive. In particular, Flannel Host-GW's high CV is driven by the cold-start effect discussed above; excluding the first two warmup runs yields a stable-state mean of 0.124 Gbps.

Table 11: TCP Throughput Statistical Summary

CNI Config	Mean (Gbps)	Stdev	CV%	Warmup-adj Mean
Flannel VXLAN	0.118	0.006	5.1	—
Flannel Host-GW	0.088	0.049	55.7	0.124
Calico VXLAN	0.119	0.015	12.6	—
Calico BGP	0.116	0.009	7.8	—
Cilium VXLAN	0.058	0.001	1.7	—
Cilium Underlay	0.102	0.009	8.8	—

6.8 Analysis and Discussion

6.8.1 The "Edge Environment" Paradox

While industry benchmarks often favor Cilium and Calico for large-scale datacenters, our 3-node WiFi cluster results demonstrate that **simplicity outperforms sophistication** in resource-constrained or high-latency environments. This finding reframes WiFi not merely as a limitation but as a distinct research contribution: prior CNI benchmarks have exclusively used wired Ethernet, leaving the behavior of CNI plugins under edge computing, university lab, IoT gateway, and development environment conditions completely uncharacterized. Our WiFi results provide the first quantitative evidence that **CNI selection criteria differ fundamentally between bandwidth-rich and bandwidth-constrained environments**: optimize for throughput in wired environments, optimize for request latency in constrained environments.

Why Flannel Wins (The Efficiency of Minimalism). Flannel's dominance in TCP_RR (request/response) and its low CPU usage (3.9%) stem from its "passive" architecture:

- **Minimal logic:** Flannel simply wraps packets and hands them to the kernel. It does not perform deep packet inspection or identity-based filtering.
- **WiFi bottleneck:** Since the WiFi medium introduces significant physical latency, the overhead of Cilium's eBPF programs or Calico's BGP path calculations becomes a "tax"

that yields no performance gain. Flannel's "dumb pipe" approach leaves more CPU headroom for the OS to manage WiFi signal jitter and retransmissions.

The Cilium Underlay Performance Penalty. The finding that Cilium Underlay (111 trans/sec) is slower than its VXLAN path in TCP_RR reveals an important technical detail:

- **Per-transaction overhead:** Cilium's eBPF programs execute complex logic (security identity checks and state tracking) for every single packet.
- **Optimization mismatch:** In the underlay routing path, the kernel must perform a full routing table lookup for every packet. In our low-throughput WiFi environment, the fixed cost of these eBPF hooks is higher than the simple encapsulation overhead of VXLAN. Effectively, the "intelligence" of Cilium is slower than "simple" encapsulation when the network cannot move fast enough to benefit from eBPF's shortcuts.

Calico's "Management Tax." The higher CPU usage for Calico (6.0%) is attributed to its control plane architecture:

- **Background noise:** Calico runs a BGP daemon (BIRD) and a policy agent (Felix). Even when the cluster is idle, these components maintain routing tables and state.
- **Scale overkill:** In a 3-node cluster, BGP is unnecessary overhead. The CPU "cost" of running a full routing protocol outweighs the benefits, which only become apparent when managing hundreds of nodes and thousands of routes.

Conclusion for edge environments: For edge, IoT, or home lab environments running on non-wired infrastructure, Flannel VXLAN remains the optimal choice. The advanced features of Cilium and Calico introduce "micro-latency" at the software level that, when combined with the "macro-latency" of WiFi, results in significantly lower transaction rates.

6.8.2 Throughput–Latency Inversion in Cilium VXLAN

A particularly striking result is that Cilium VXLAN has the *worst* TCP throughput (0.058 Gbps) but the *best* HTTP P50/P99/Max latency (6.56/27.75/38.72 ms). This apparent contradiction reveals a fundamental architectural distinction:

- **TCP throughput** measures sustained bandwidth, where per-packet overhead accumulates *multiplicatively*. Each packet traverses the eBPF VXLAN encapsulation path, and Cilium's connection tracking maps and policy enforcement run on every packet regardless of whether policies are configured. At WiFi speeds, this per-packet tax halves effective bulk throughput.
- **HTTP latency** measures individual request-response cycles, where eBPF's O(1) connection tracking lookup and iptables bypass provide a *per-request* advantage regardless of link speed. Each HTTP transaction benefits from Cilium's fast-path regardless of the underlying bandwidth.

Practical implication: Organizations running microservice workloads on constrained networks should prefer Cilium VXLAN despite its bandwidth penalty. The P99 advantage (27.75 ms vs. 38–111 ms for others) and Max advantage (38.72 ms vs. 188–682 ms) are critical for meeting SLA targets in latency-sensitive applications.

6.8.3 UDP Comparison Caveat

While the UDP results provide useful relative rankings, direct cross-CNI comparison is limited by the high packet loss rates (38–68% overall). At the 10 Mbps target—the only rate where the WiFi link is not saturated—only Calico BGP and Cilium Underlay achieve near-perfect delivery (<0.15% loss). The remaining CNIs exhibit 4.7–43% loss even at this gentle rate, suggesting that their packet-processing pipelines introduce sufficient per-packet delay to overflow kernel socket buffers on a jittery WiFi link. The 25 Mbps and 50 Mbps targets drive all CNIs well past the WiFi medium's capacity, making these rates less informative for CNI differentiation and more a measure of WiFi saturation behavior.

6.8.4 CPU Overhead: Deeper Interpretation

At 0.119 Gbps with 1500-byte MTU, the node processes approximately 10,000 packets/second—well below the millions/second threshold where eBPF vs.
iptables differences manifest. This explains why all CNIs show similar CPU usage (3.9–6.0%): the WiFi medium rate-limits packets to a level where software processing overhead is negligible.

The Calico VXLAN kube-apiserver spike (135.5 mcpu vs. 28–34 mcpu for others) likely reflects the Felix daemon polling the apiserver for policy updates during test initialization rather than data-plane overhead. This is an architectural difference: **Calico's control plane is tightly coupled to the apiserver**, whereas Cilium and Flannel operate more independently after initial configuration.

6.8.5 Cross-Environment Comparison

Comparing with the kind-cluster results reveals important environmental effects:

Table 12: Kind Cluster vs. Bare-Metal (WiFi) — Throughput Comparison

CNI	Kind (Gbps)	Bare-Metal (Gbps)	Ratio
Flannel VXLAN	30.96[a]	0.118	262×
Calico VXLAN	23.83[a]	0.119	200×
Cilium VXLAN	27.40[a]	0.058	472×

[a]Kind-cluster T2 cross-node, 1 MB, $c = 16$ (representative middle-ground configuration).

The 200–472× throughput gap confirms that the kind-cluster's Docker bridge is far more efficient than a physical WiFi link. Notably, Cilium's ratio (472×) is much larger than Flannel's (262×), indicating that Cilium's eBPF VXLAN overhead is amplified on constrained physical networks.

The **relative rankings partially shift** between environments: in the kind-cluster, Cilium leads in throughput via eBPF fast-path advantages; in bare-metal WiFi, Cilium VXLAN trails because the per-packet overhead matters more when the link is slow. This highlights the importance of benchmarking on representative hardware rather than relying solely on simulated environments.

7. Related Work

- **Qi et al. (2021)** benchmarked CNI plugins including Flannel, Calico, and Cilium on bare-metal and cloud environments, finding Calico BGP fastest on bare-metal. Our results partially confirm this—Calico BGP performs well on WiFi bare-metal—but also show that the physical network medium can dominate CNI-level differences.

- **Part A (Kind-Cluster Report)** provides the companion intra-node evaluation, enabling direct comparison between simulated and physical network environments. The cross-environment comparison (Section 6.7) reveals how CNI performance rankings shift with underlying hardware.
- **Cilium benchmarking documentation** reports eBPF throughput advantages, which we confirm for the kind-cluster but find inverted when Cilium's VXLAN path is constrained by WiFi bandwidth.
- This study extends prior work by systematically comparing overlay (VXLAN) vs. underlay routing modes for *all three* CNIs on the same hardware, a comparison not previously available in the literature.

8. Outcomes and Conclusion

This study presents a performance comparison of six CNI configurations (3 plugins $\times$ 2 routing modes) on a bare-metal Kubernetes cluster connected over WiFi, comprising 90+ benchmark executions across iperf3 (TCP and UDP), netperf, and Fortio, plus CPU/memory overhead monitoring.

8.1 Key Findings

1. **WiFi is the dominant bottleneck:** All CNI configurations except Cilium VXLAN converge to a narrow 0.088–0.119 Gbps throughput band, indicating that the physical medium—not the CNI—limits bare-metal performance on constrained networks.
2. **Overlay vs. underlay routing is CNI-dependent:** There is no universal winner. Flannel Host-GW $\geq$ VXLAN (once warm), Calico VXLAN $\approx$ BGP, and Cilium Underlay $\gg$ VXLAN. The architectural specifics of each CNI's encapsulation and routing paths determine which mode is faster.
3. **Cilium VXLAN shows a throughput–latency paradox:** Despite having the lowest throughput (0.058 Gbps), Cilium VXLAN achieves the best HTTP latency across all percentiles (P50 = 6.56 ms, Max = 38.72 ms). eBPF's fast-path excels at individual request processing even when bulk encapsulation is costly.
4. **Flannel's simplicity wins on latency:** Flannel configurations deliver the highest TCP_RR transaction rates (190–214 trans/sec), confirming that minimal per-packet processing yields the best results when the network is slow.
5. **Calico BGP dominates UDP:** With 11.88 Mbps throughput and 38% packet loss, BGP's underlay routing avoids encapsulation overhead on connectionless traffic. Calico VXLAN is the worst (5.78 Mbps, 68% loss), revealing that iptables-based VXLAN processing creates a severe bottleneck for UDP.
6. **CPU overhead is uniformly modest:** Node-level utilization ranges from 3.9% (Flannel VXLAN) to 6.0% (Calico BGP), confirming that WiFi—not CPU—is the bottleneck. Flannel's lightweight architecture yields the lowest resource footprint.
7. **Underlay routing introduces cold-start risks:** Flannel Host-GW and (to a lesser extent) Calico BGP exhibit warmup effects on WiFi, where initial runs underperform until ARP caches and routing tables stabilize.

8. **CNI rankings shift with hardware:** Cilium leads throughput in kind-cluster (63.98 Gbps same-node) but trails in bare-metal WiFi (0.058 Gbps VXLAN). Production CNI selection must be validated on representative hardware.

8.2 Recommendations

CNI Selection for Bare-Metal Clusters

- **Low-bandwidth or constrained networks:** Flannel VXLAN—consistent TCP performance with lowest latency, lightest CPU footprint (3.9%), and simplest operations.
- **Latency-sensitive microservices:** Cilium VXLAN—best HTTP P50/P99/Max despite lower throughput; eBPF efficiently handles request-response workloads.
- **UDP-heavy workloads:** Calico BGP—highest UDP throughput (11.88 Mbps), lowest loss at moderate rates (0.11% at 10 Mbps), and underlay routing eliminates encapsulation overhead for connectionless traffic.
- **Throughput-critical workloads on wired networks:** Calico VXLAN or Cilium Underlay—both saturate available bandwidth with reasonable stability. Cilium Underlay offers +77% over Cilium VXLAN when the link supports it.
- **Security-conscious environments:** Calico—proven NetworkPolicy enforcement and BGP peering for production-grade network segmentation.

Table 13: CNI Decision Matrix — Quick Reference

Use Case	Recommended	Rationale
Constrained/WiFi network	Flannel VXLAN	Lowest CPU (3.9%), consistent throughput
Latency-critical services	Cilium VXLAN	Best P99 (27.75 ms), lowest Max (38.72 ms)
High throughput (wired)	Cilium Underlay	+77% over VXLAN on bare-metal
UDP-heavy workloads	Calico BGP	Best UDP throughput, <0.15% loss at 10 Mbps
Security-critical	Cilium	Lower WireGuard overhead (+365% vs. +537% Calico)
Large clusters	Cilium KPR	Bypasses iptables rule explosion at scale
Lowest complexity	Flannel VXLAN	No BGP/eBPF, minimal control plane

8.3 Limitations

- **WiFi medium:** The shared wireless link introduces variance and limits absolute throughput to ≈0.12 Gbps. Wired (1–100 GbE) tests would better differentiate CNI processing overhead. However, the WiFi environment is also a deliberate research contribution: prior CNI benchmarks exclusively use wired Ethernet, leaving CNI behavior under edge computing, university lab, IoT gateway, and development environment conditions uncharacterized. Our results provide the first quantitative evidence that CNI selection criteria

differ between bandwidth-rich and bandwidth-constrained environments.

- **No extension experiments:** Unlike Part A, this study does not test NetworkPolicy enforcement, WireGuard encryption, or kube-proxy replacement. These remain areas for future bare-metal evaluation.
- **Limited payload sizes for throughput:** iperf3 tests use default settings (single stream, 30 s). Varying payload sizes and concurrency levels (as done in Part A) would provide more granular throughput characterization.
- **Calico BGP with 4 runs:** One TCP throughput run failed, leaving 4 data points instead of 5. While the existing data is consistent, the smaller sample size slightly reduces statistical confidence.
- **Node heterogeneity:** The three laptops may have different WiFi chipsets and antenna configurations, introducing asymmetric link quality that is not controlled for.
- **Cilium VXLAN UDP anomaly:** One UDP run at 50 Mbps recorded 0.00 Mbps throughput with 0% loss, likely a failed/stalled run rather than a genuine measurement.
- **CPU sampling granularity:** `kubectl top` provides 2-second averages from the metrics-server, which may smooth over sub-second CPU spikes during burst processing.

8.4 Future Work

- Repeat benchmarks on wired Ethernet (1 GbE, 10 GbE) to isolate CNI processing overhead
- Add extension experiments (NetworkPolicy, WireGuard, KPR) on bare-metal
- Test with multi-stream iperf3 and varied payload sizes
- Profile per-packet CPU cost using perf/eBPF tracing during high-throughput tests
- Deploy application-level benchmarks (Redis, NGINX, gRPC) to evaluate CNI impact on real workloads
- Test UDP at higher sustained rates on wired networks where WiFi is not the limiting factor

References

[1] Flannel Project. “flannel—Network fabric for containers.” https://github.com/flannel-io/flannel

[2] Project Calico. “Calico Networking for Kubernetes.” https://docs.tigera.io/calico/latest/about

[3] Cilium Project. “Cilium—eBPF-based Networking, Security, and Observability.” https://cilium.io/

[4] iperf3. “A TCP, UDP, and SCTP network bandwidth measurement tool.” https://iperf.fr/

[5] Netperf. “Network Performance Benchmark.” https://hewlettpackard.github.io/netperf/

[6] Fortio. “Load testing library, command line tool, and web UI.” https://fortio.org/

[7] Qi, S. et al. (2021). "Assessing Container Network Interface Plugins: Functionality, Performance, and Scalability." *IEEE Transactions on Network and Service Management*, 18(1).

[8] Kubernetes Documentation. "Cluster Networking." https://kubernetes.io/docs/concepts/cluster-administration/networking/

[9] VXLAN RFC 7348. "Virtual eXtensible Local Area Network." https://datatracker.ietf.org/doc/html/rfc7348